Architecture Comparison: Modular Monolith vs. Microservices

This document compares the current **Modular Monolith** architecture of DocPilot with a **Microservices** alternative.

1. Patterns Overview

Implemented: Modular Monolith (Layered)

The current DocPilot backend follows a layered approach where the `api`, `services`, and `infrastructure` layers reside in a single process.

- **Communication**: In-process function calls.
- **Data Store**: Single PostgreSQL database with PGVector.
- **Dependency Management**: Centralized `pyproject.toml`.

Alternative: Microservices Architecture

In this pattern, DocPilot would be split into independent services:

1. **Ingestion Service**: Handles file uploads and OCR.
2. **Embedding Service**: Processes chunks into vectors.
3. **Search Service**: Interfaces with PGVector for retrieval.
4. **Query Service**: Handles synthesis and LLM routing.

- **Communication**: REST/gRPC or Message Queues (e.g., RabbitMQ).
- **Data Store**: Database-per-service or shared vector cluster.

---

2. Quantitative NFR Comparison

The following quantifications are based on typical performance benchmarks for Python/FastAPI applications and external LLM integrations.

Metric	Modular Monolith (Current)	Microservices (Alternative)
Response Time (Overhead)	**~5ms - 15ms**	**~150ms - 400ms**
Max Throughput (Ingestion)	**Linear Scaling (Full App)**	**Independent Scaling (High)**
Deployment Time	**2-5 Minutes**	**10-30 Minutes (Orchestrated)**
Fault Isolation	**Low (Process Crash = Downtime)**	**High (Isolated Failures)**

NFR 1: Response Time (Latency)

In a **Modular Monolith**, the `RagEngine` calls the `DocumentRepository` directly. This is a memory-address jump taking microseconds.
In a **Microservices** model, a query must cross the network:

1. API Gateway → Query Service (30ms)
2. Query Service → Search Service (40ms)
3. Search Service → DB (10ms)
4. Serialization (JSON/Protobuf) adds ~20ms per hop.
   Quantification: The monolith saves approximately **200ms+** per request by avoiding network I/O between internal components.

NFR 2: Throughput (Scalability)

If DocPilot receives 10,000 documents for ingestion, the **Monolith** must scale the entire backend process.
In **Microservices**, we can spin up 50 instances of the `Embedding Service` while keeping only 2 instances of the `Query Service`.
Quantification: Microservices can achieve **3x - 5x higher throughput** for specific bottlenecks (like CPU-intensive embedding or OCR) without wasting memory on idle components.

---

3. Trade-off Analysis

Why Modular Monolith was Chosen

1. **Development Velocity**: No need to manage service contracts (OpenAPI/Protobuf) between internal modules.
2. **Reduced Cognitive Load**: Single repository and simplified debugging (stack traces aren't split across logs).
3. **Operational Simplicity**: Deployment involves a single Docker container.

When to Switch to Microservices

- **Team Size**: If >10 developers are working on DocPilot, merge conflicts and deployment coordination will become a bottleneck.
- **Heterogeneous Tech Stack**: If the Embedding engine needs Python (for PyTorch) but the API wants Go (for performance).
- **Extreme Scale**: If document ingestion needs to handle millions of pages per hour.

Conclusion

The **Modular Monolith** is the optimal choice for DocPilot's current stage, prioritizing **low latency** and **developer productivity** over the granular scalability of Microservices.